

Chapter Five

System Implementation: Coding and Testing

Introduction

In modern software engineering work, coding may be

- the **direct creation** of programming language source code (e.g., Java),
- the **automatic generation of source code** using an intermediate design-like representation of the component to be built, or
- the **automatic generation of executable code** using a “fourth-generation programming language” (e.g., Visual C++).

Coding Principles

- The principles that guide the coding task are closely aligned with **programming style, programming languages, and programming methods.**
- However, there are a number of fundamental principles that can be stated

Preparation principles:

Before you write one line of code, be sure you

- Understand of the **problem** you're trying to solve.
- Understand basic **design principles and concepts**.
- Pick a **programming language** that meets the needs of the software to be built and the environment in which it will operate.
- Select a **programming environment** that provides **tools** that will make your work easier.
- Create a set of **unit tests** that will be applied once the component you code is completed.

Programming principles: As you begin writing code, be sure you

- Constrain your **algorithms** by following structured programming practice.
- Consider the use of pair programming.
- Select **data structures** that will meet the needs of the design.
- Understand the **software architecture** and create interfaces that are consistent with it.
- Keep conditional logic as **simple as possible**.

Cont..

- Create **nested loops** in a way that makes them easily testable.
- Select **meaningful variable names** and follow other local coding standards
- Write code that is **self-documenting**.
- Create a **visual layout** (e.g., indentation and blank lines) that aids understanding.

Validation Principles:

After you've completed your first coding pass , be sure you

- Conduct a code **walkthrough** when appropriate.
- Perform **unit tests** and **correct errors** you've uncovered.
- Refactor the code

Goal: Self-Documenting Code

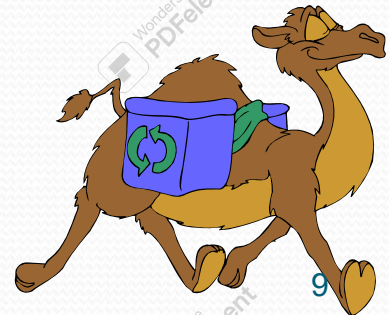
- **Self-documenting** explains *itself* without need for external documentation, like flowcharts, UML diagrams, process-flow diagrams, etc.
 - *Doesn't imply we don't like/use those documents!*
- **Coding conventions** target:
 - How you write **statements** in the language, **organize** them into “**modules**,” **format** them in the source files
 - *Module: generic term meaning C function, Java/C++ class, etc.*
 - How you create **names**
 - How you write **comments**

Standard Coding Practices

- Teams strive to use the same **coding conventions** in every regard:
 - **Name** your classes similarly, your variables, your functions.
 - **Comment** the same way, **format** your code the same way.
 - By doing this, you ensure rapid understanding of **whatever module needs changing**, and as they evolve, your modules will not degenerate into a *HorseByCommittee* appearance.



8/28/2014



Benefits

- Projects **benefit** from having strong Coding Conventions/Standards because...
 - People can stop **reformatting** code and **renaming** variables and methods whenever working on code written by **other** people.
 - It's slightly **easier to understand** code that is consistently formatted and uses a consistent naming standard.
 - It's **easier to integrate** modules that use a common consistent naming standard -- less need to look up and cross-reference the different names that refer to the same thing.

Comments and code conventions

- Comments commonly use to describe the followings:
 - File headers
 - Function headers
 - Explanations of variables and statements
- Names (chosen by programmer)
- Statements
 - Organization: files, “modules,” nesting
 - Format: spacing and alignment

Organization of Program

- *Analogy*: Organize programs for **readability**, as you would organize a book:
 - Title page & Table of contents → File header
 - Chapter → Module (function or logical group of functions)
 - Paragraph → Block of code
 - Index & Glossary → can be generated automatically if comments are used wisely (**Javadoc**, **doxygen**)
 - Cross references → **ctags**.sourceforge.net free tool

Organization of Modules

- Apply principle of **information hiding**
 - Hide details of implementation that users don't need to know.
- Divide each module into a **public** part and a **private** part.
 - public part goes into an *include* (.h) file
 - private part goes into a *source* (.c) file

File Headers

Tells at a glance what file you're looking at and what's in there.

- *Filename*

- Suggests the purpose of the file's module(s)
- Could be class name (Java/C++)

- *Description*

- This is the main reason to have the header. It clearly tells the purpose of the module(s), their role in the system. If you can't describe the purpose in one or two sentences, you've thrown unrelated modules together.

File Headers

- *Creation date*
 - Provides a creation timestamp for copyright purposes, but it does more than that. It provides a quick clue to the *context* that existed at the time the module was created. Not as accurate as **source control**, but *quick* and *maintenance free*.
- *Author's Name or Initials*
- *Copyright banner*
 - This identifies the uses to which this code can be put.

File Headers

- *E-mail address*
 - If this is publicly released code, then the author should be contactable.
- *The lack of anything else*
 - More info likely has to be kept up to date.
 - Maintenance free, *simplicity* and *clarity* are the goals.

Function Headers

- Commenting **interfaces** is an especially good thing to do.
 - **Function headers** describe purpose of function and use of arguments, return value, also pre-/post-conditions.
 - public interface function prototypes go in .h files
 - internal helper functions in .c files (unless called from multiple .c files)
- **Implementations** don't need to be commented heavily if their interfaces are well-documented.
 - A well-written implementation for a given function is rarely more than 100 language statements. Thus, if you know what the function is supposed to do, it should be straightforward to understand the mechanics of it.

Meaningful Comments

- **Comments** provide **meta** information about the program, reasons for choosing this algorithm or implementation, known issues, hints for **future readers** (including yourself).
- “Meaningful” in this context has two parts:
 - The comment can be **understood** by readers.
 - The comment says something that is **likely not to be understood** by the same readers unless it was present.
 - Don’t “parrot the code”:

```
x = 2 + y;    // add 2 to y
```
- Avoid comments needing heavy maintenance

Variable Names

- Use simple, descriptive variable names.
- Good names can be created by using one word or putting multiple words together joined by underscores or caps
 - prefer usual English word order

```
#define MAX_FIELD 127  
int numStudents, studentID;  
char *homeAddr;
```

Variable Names

- Follow every variable declaration with a comment that defines it.
- Group similar variables together.
 - use similar names for variables that perform similar functions

Statement Style

- Put each statement on a line by itself.
- Avoid very long statements. Use two shorter statements instead.
- Keep all lines to 80 characters or less.
- Group statements in logical “chunks” separated with white space (blank lines)
 - Helps eye follow logic without getting overwhelmed

Layout of Source Code

- Good layout affects how easy it is to understand the code, review it, and revise it months after it was written
- Picky details?
 - Attention to such details makes a difference in the quality and maintainability of the code!
 - Must be done during initial construction
- Team should agree on a style before coding begins

Scientific Debugging

- For programming
 - Identify the error
 - Locate the source of the error
 - Fix the error
 - Test the fix
 - Look for similar errors

Tips on Finding Errors

- Refine the test cases that produce the error
- Reproduce the error several different ways
- Narrow the suspicious/doubtful region of code
- Be suspicious of routines that have had errors before
- Check code that's changed recently
- Integrate incrementally
- Check for common errors
- Talk to someone else about the problem
- Take a break

Tips of Fixing Errors

- Understand the problem before you fix it
- Understand the program, not just the problem
- Confirm the error diagnosis
- Relax
- Save the original source code
- Fix the problem, not the symptom
- Make one change at a time
- Check your fix
- Look for similar errors

Code-Tuning Strategies

- Code tuning is the practice of **modifying** correct code to make it run more efficiently.
- Less of a factor in many of today's systems, particularly business software.
- Problem: *Efficient code isn't necessarily better code*

Some are better than others

- Study of twelve programmers with at least four years of experience
- Fastest three programmers:
 - Average debug time: 5 minutes
 - Average number of errors not found: 0.7
 - Average number of errors made correcting errors: 3.0
- Slowest three programmers:
 - Average debug time: 14.1 minutes
 - Average number of errors not found: 1.7
 - Average number of errors made correcting errors: 7.7
- Use errors as opportunities
 - Learn about the program, kinds of mistakes you make, and how you fix errors.

Communication and Cooperation

- Truly excellent programmers learn how to work and play with others
 - This includes writing readable code
- Most good programmers enjoy making programs readable, given enough time, although there are a few holdouts
 - Level 1: Beginner
 - Capable of using **basic capabilities**, e.g. loops, conditionals, write routines
 - Level 2: Intermediate
 - Capable of basic routines of **multiple languages**
 - Level 3: Specialist
 - Expertise in a **language or environment or both**, many stuck here
 - Level 4: Guru
 - Level 3 plus recognizes 85% of programming is communicating with other people
 - Only 30% of an programmer's time is spent working alone, on average
 - **Guru writes crystal clear code, documents it, results in guru status**

Observations about Testing

- “Testing is the process of executing a program with the intention of finding errors.” – Myers
- “Testing can show the presence of bugs but never their absence.” - Dijkstra

Characteristics of Testable Software

- Operable
 - The better it works (i.e., better quality), the easier it is to test
- Observable
 - Incorrect output is easily identified; internal errors are automatically detected
- Controllable
 - The states and variables of the software can be controlled directly by the tester
- Decomposable
 - The software is built from independent modules that can be **tested independently**

Characteristics of Testable Software (continued)

- Simple
 - The program should exhibit functional, structural, and code simplicity
- Stable
 - Changes to the software during testing are infrequent and do not invalidate existing tests
- Understandable
 - The architectural design is well understood; documentation is available and organized

Good Testing Practices

- A good test case is one that has a high probability of **detecting an undiscovered defect**, not one that shows that the program works correctly.
- It is **impossible to test your own program**
- A necessary part of every test case is a description of the expected result

Good Testing Practices (cont'd)

- Write test cases for valid as well as invalid input conditions.
- Thoroughly investigate the results of each test
- As the number of detected defects in a piece of software increases, the probability of the existence of more undetected defects also increases.

Good Testing Practices (cont'd)

- Assign your best people to testing
- Ensure that testability is a key objective in your software design
- Never alter the program to make testing easier
- Testing, like almost every other activity, *must start with objectives*

Test Characteristics

- A good test has a high probability of **finding an error**
 - The tester must understand the software and how it might fail
- A good test is **not redundant**
 - Testing time is limited; one test should not serve the same purpose as another test
- A good test should be “***best of other tests***”
 - Tests that have the highest likelihood of uncovering a whole class of errors should be used
- A good test should be **neither too simple nor too complex**
 - Each test should be executed separately; combining a series of tests could cause side effects and mask certain errors

Two Unit Testing Techniques

- **Black-box testing**

- Knowing the **specified function** that a product has been **designed to perform**, test to see if that function is fully operational and error free
- Includes tests that are conducted at the software interface
- Not concerned with internal logical structure of the software

- **White-box testing**

- Knowing the **internal workings of a product**, test that all **internal operations** are performed according to specifications and all internal components have been exercised
- Involves tests that concentrate on close **examination of procedural detail**.
- Logical paths through the software are tested
- Test cases exercise specific **sets of conditions and loops**

Classification of testing techniques

- Classification based on the criterion to measure the adequacy of a set of test cases:
 - coverage-based testing
 - fault-based testing
 - error-based testing
- Classification based on the source of information to derive test cases:
 - black-box testing (functional, specification-based)
 - white-box testing (structural, program-based)

Cont.

- Coverage-based: e.g. how many statements or requirements have been tested so far
- Fault-based: e.g., how many seeded faults are found
- Error-based: focus on error-prone points, e.g. off-by-one points
- Black-box: **you do not look inside**, but only base yourself on the specification/functional description
- White-box: **you do look inside**, to the structure, the actual program/specification.

Some preliminary questions

- What exactly is an error?
- How does the testing process look like?
- When is test technique A superior to test technique B?
- What do we want to achieve during testing?
- When to stop testing?

Error, fault, failure



- an *error* is a human activity resulting in software containing a fault
- a *fault* is the manifestation of an error
- a *fault* may result in a failure

When exactly is a failure ?

- Failure is a relative notion: e.g. a failure w.r.t. the specification document

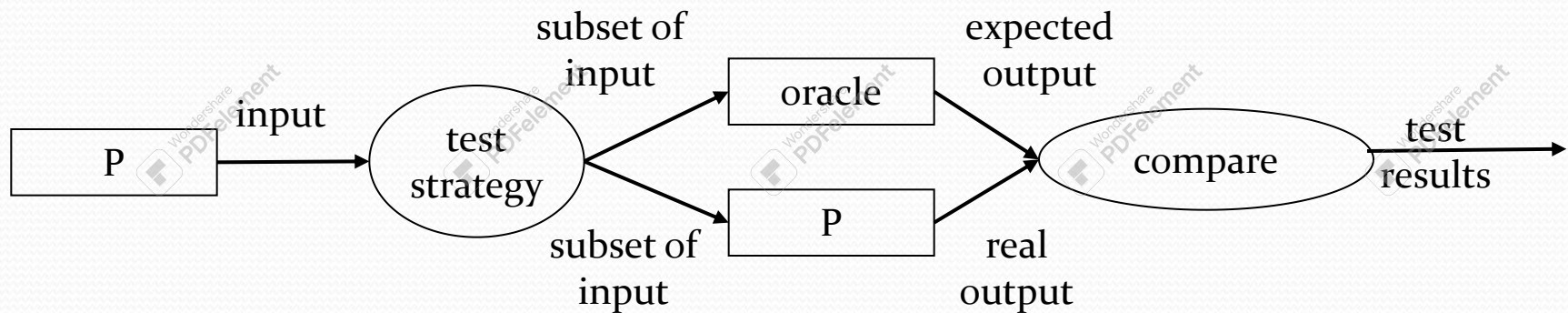
- *Verification*: evaluate a product to see **whether it satisfies the conditions specified at the start**:

Have we built the system right?

- *Validation*: evaluate a product to see **whether it does what we think it should do**:

Have we built the right system?

Testing process



Test adequacy criteria

- Specifies requirements for testing
- Can be used as *stopping rule*: stop testing if 100% of the statements have been tested
- Can be used as *measurement*: a test set that covers 80% of the test cases is better than one which covers 70%
- Can be used as *test case generator*: look for a test which exercises some statements not covered by the tests so far
- A given test adequacy criterion and the associated test technique are opposite sides of the same coin

What is our goal during testing?

- Objective 1: find as many faults as possible
- Objective 2: make you feel confident that the software works OK



Testing and the life cycle

- requirements engineering

- criteria: completeness, consistency, feasibility, and testability.
- typical errors: missing, wrong, and extra information
- determine testing strategy
- generate functional test cases
- test specification, through reviews and the like

- design

- functional and structural tests can be devised on the basis of the decomposition
- the design itself can be tested (**against the requirements**)
- formal verification techniques
- the architecture can be evaluated

Testing and the life cycle (cnt'd)

- implementation
 - check consistency of implementation and previous documents
 - code-inspection and code-walkthrough
 - all kinds of functional and structural test techniques
 - extensive tool support
 - formal verification techniques
- maintenance
 - regression testing: either retest all, or a more selective retest

Levels of Testing

- Unit Testing
- Integration Testing
- Validation Testing
 - Regression Testing
 - Alpha Testing
 - Beta Testing
- Acceptance Testing

Unit Testing

- Algorithms and logic
- Data structures (global and local)
- Interfaces
- Independent paths
- Boundary conditions
- Error handling

Why Integration Testing Is Necessary

- One module can have an adverse effect on another sub-functions, **when combined, may not produce the desired major function**
- Interfacing errors not detected in unit testing may appear

Why Integration Testing Is Necessary (cont'd)

- Timing problems (in real-time systems) are not detectable by unit testing
- Resource contention problems are not detectable by unit testing

Top-Down Integration

1. The main control module is used as a driver, and stubs are substituted for all modules directly subordinate to the main module.
2. Depending on the integration approach selected (depth or breadth first), subordinate stubs are replaced by modules one at a time.

Top-Down Integration (cont'd)

3. Tests are run as each individual module is integrated.
4. On the successful completion of a set of tests, another stub is replaced with a real module
5. Regression testing is performed to ensure that errors have not developed as result of integrating new modules

Problems with Top-Down Integration

- Many times, calculations are performed in the modules at the bottom of the hierarchy Stubs typically do not pass data up to the higher modules.
- Delaying testing until lower-level modules are ready usually results in *integrating many modules at the same time rather than one at a time*
- Developing stubs that can pass data up is almost as much work as developing the actual module

Bottom-Up Integration

- Integration begins with the lowest-level modules, which are combined into clusters, or builds, that perform a specific software subfunction
- Drivers (control programs developed as stubs) are written to coordinate test case input and output
- The cluster is tested
- Drivers are removed and clusters are combined moving upward in the program structure

Problems with Bottom-Up Integration

- The whole program does not exist until the last module is integrated
- Timing and resource contention problems are not found until late in the process

Validation Testing

- Determine if the software meets all of the requirements defined in the SRS
- Regression testing is performed to determine if the software still meets all of its requirements in light of changes and modifications to the software
- Regression testing involves selectively repeating existing *validation tests, not developing new tests*

Alpha and Beta Testing

- It's best to provide customers with an *outline of the things* that you would like them to **focus on and specific test scenarios for them to execute.**
- Provide with **customers** who are actively involved with **a commitment to fix defects that they discover.**

Acceptance Testing

- Similar to validation testing **except** that **customers are present or directly involved.**
- Usually the tests are developed by *the customer.*

System Testing

- Recovery testing
 - checks system's **ability to recover from failures**
- Security testing
 - verifies that system **protection** mechanism prevents improper penetration or data alteration
- Stress testing
 - program is checked to see **how well** it deals with abnormal resource demands
- Performance testing
 - tests **the run-time performance** of software

Test Planning

- **The Test Plan** – defines the *scope of the work* to be performed
- **The Test Procedure** – a container document that holds all of the **individual tests** (test scripts) that are to be *executed*
- **The Test Report** – documents what occurred when the *test scripts were run*.

Test Plan

- *Questions to be answered:*
 - How many tests are needed?
 - How long will it take to develop those tests?
 - How long will it take to execute those tests?
- *Topics to be addressed:*
 - Test estimation
 - Test development and informal validation
 - Validation readiness review and formal validation
 - Test completion criteria

Test Estimation

- Number of test cases required is based on:
 - Testing all functions and features in the SRS
 - Including an appropriate number of ALAC (*Act Like A Customer*) tests including:
 - Do it wrong
 - Use wrong or illegal combination of inputs
 - Don't do enough
 - Do nothing
 - Do too much
 - Achieving some test coverage goal
 - Achieving a software reliability goal

Considerations in Test Estimation

- **Test Complexity** – It is better to have many small tests than a few large ones.
- **Different Platforms** – Does testing need to be modified for **different platforms, operating systems**, etc.
- **Automated or Manual Tests** – Will automated tests be developed? Automated tests take more time to create but do not require human intervention to run.

Test Team Members

- Professional testers.
- Analysts.
- System designers.
- Configuration management specialists.
- Users.

Debugging

- Debugging (**removal of a defect**) occurs as a consequence of successful testing.
- Some people are better at debugging than others.
- Is the cause of the bug reproduced in another part of the program?
- What “next bug” might be introduced by the fix that is being proposed?
- What could have been done to prevent this bug in the first place?

Test Report

- Completed *copy of each test script with evidence* that it was executed (i.e., dated with the signature of the person who ran the test)
- *Software Problem Reports (SPRs)* are submitted for each test that fails.
- Copy of each SPR (Software Problem Reports) showing resolution
- List of open or unresolved SPRs
- Identification of SPRs found in each baseline along with total number of SPRs in each baseline
- Regression tests executed for each software baseline

Thank You!
Q?